

LAPORAN PRAKTIKUM 3
STRUKTUR DATA
IMPLEMENTASI *ABSTRACT DATA TYPE (ADT) STACK*
MENGGUNAKAN *ARRAY & ARRAYLIST*



Disusun Oleh:

Naysha Aprilia Rizki

2511531014

Dosen Pengampu:

Wahyudi. Dr. S.T.M.T

Asisten Praktikum:

Muhammad Galid

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum yang berjudul “Laporan Praktikum 3 Struktur Data: Implementasi Abstract Data Type (ADT) Stack Menggunakan Array dan ArrayList” ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum pada mata kuliah Struktur Data. Adapun tujuan dari praktikum ini adalah untuk memahami konsep Abstract Data Type (ADT), khususnya pada struktur data Stack, serta mengimplementasikannya menggunakan Array dan ArrayList dalam bahasa pemrograman Java.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca.

Padang, 17 April 2026

Penulis

BAB I

PENDAHULUAN

1.1. Latar Belakang

Struktur data merupakan salah satu konsep penting dalam ilmu komputer yang digunakan untuk mengelola dan menyimpan data secara efisien. Pemilihan struktur data yang tepat akan sangat berpengaruh terhadap kinerja suatu program, baik dari segi waktu maupun penggunaan memori.

Salah satu bentuk struktur data yang sering digunakan adalah Stack. Stack merupakan struktur data yang menerapkan prinsip LIFO (*Last In First Out*), yaitu elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Konsep ini banyak digunakan dalam berbagai kasus, seperti pengolahan ekspresi matematika, mekanisme undo-redo, serta pemanggilan fungsi dalam program.

Dalam implementasinya, Stack dapat dibuat menggunakan berbagai cara, di antaranya menggunakan Array dan ArrayList. Array memiliki ukuran tetap, sedangkan ArrayList bersifat dinamis sehingga lebih fleksibel dalam pengelolaan data. Oleh karena itu, penting untuk memahami bagaimana cara mengimplementasikan Abstract Data Type (ADT) Stack menggunakan kedua struktur tersebut serta mengetahui perbedaan karakteristiknya.

Berdasarkan hal tersebut, praktikum ini dilakukan untuk mempelajari dan mengimplementasikan ADT Stack menggunakan Array dan ArrayList dalam bahasa pemrograman Java.

1.2. Tujuan

1. Memahami konsep dasar Abstract Data Type (ADT).
2. Memahami prinsip kerja struktur data Stack (LIFO).
3. Mengimplementasikan Stack menggunakan Array.
4. Mengimplementasikan Stack menggunakan ArrayList.
5. Membandingkan penggunaan Array dan ArrayList dalam implementasi Stack.

5.1. Manfaat

1. Menambah pemahaman mengenai konsep struktur data, khususnya Stack.
2. Melatih kemampuan dalam mengimplementasikan ADT menggunakan bahasa pemrograman Java.
3. Memahami perbedaan penggunaan Array dan ArrayList dalam pengelolaan data.
4. Meningkatkan kemampuan logika dan pemecahan masalah dalam pemrograman.
5. Menjadi dasar dalam pengembangan program yang lebih kompleks di masa mendatang

BAB 2 PEMBAHASAN

Dalam pemrograman, struktur data merupakan cara untuk menyimpan dan mengorganisasi data agar dapat digunakan secara efisien. Salah satu konsep penting dalam struktur data adalah Abstract Data Type (ADT). ADT adalah suatu tipe data abstrak yang mendefinisikan sekumpulan nilai dan operasi yang dapat dilakukan terhadap data tersebut tanpa menjelaskan bagaimana implementasinya secara detail.

Konsep utama dari ADT adalah pemisahan antara *interface* (apa yang dapat dilakukan) dan *implementasi* (bagaimana cara melakukannya). Dengan menggunakan ADT, programmer dapat lebih fokus pada fungsi dan kegunaan suatu tipe data tanpa harus memahami detail internalnya. Hal ini membuat program menjadi lebih modular, mudah dipelihara, dan mudah dikembangkan.

Dalam bahasa pemrograman Java, ADT biasanya direpresentasikan menggunakan class. Sebuah class berisi atribut (data) dan metode (fungsi) yang mendefinisikan perilaku dari objek tersebut. Atribut digunakan untuk menyimpan data, sedangkan metode digunakan untuk melakukan operasi terhadap data tersebut.

Pada praktikum ini, ADT yang diimplementasikan adalah ADT Stack menggunakan struktur data Array dan ArrayList. Stack merupakan struktur data linear yang menerapkan prinsip LIFO (*Last In First Out*), yaitu elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Struktur ini digunakan untuk mengelola data dengan mekanisme penambahan dan penghapusan yang hanya dilakukan pada satu sisi, yaitu bagian atas (*top*).

Selain itu, ADT Stack juga dilengkapi dengan beberapa operasi, antara lain:

1. Penambahan data (*push*), yaitu menambahkan elemen ke bagian atas stack.
2. Penghapusan data (*pop*), yaitu menghapus elemen dari bagian atas stack.
3. Melihat elemen teratas (*peek/top*), yaitu menampilkan elemen paling atas tanpa menghapusnya.
4. Pengecekan kondisi (*isEmpty* dan *isFull*), yaitu untuk mengetahui apakah stack dalam keadaan kosong atau penuh (khusus pada array).

Dalam implementasinya, Stack dapat dibuat menggunakan Array yang bersifat statis maupun ArrayList yang bersifat dinamis. Penggunaan Array memberikan keunggulan dalam efisiensi memori, sedangkan ArrayList memberikan fleksibilitas dalam pengelolaan jumlah data.

Dengan memahami dan mengimplementasikan ADT Stack menggunakan Array dan ArrayList, mahasiswa dapat mengembangkan kemampuan dalam mengelola data secara terstruktur serta meningkatkan pemahaman terhadap konsep dasar struktur data dalam pemrograman Java.

1. Langkah kerja

1.1 Latihan Stack_2511531014

- Kode Program

```
1 package pekan3_2511531014;
2 import java.util.Stack;
3 public class LatihanStack_2511531014 {
4     public static void main (String[] args) {
5         Stack<Integer> s = new Stack<Integer>();
6         s.push(42);
7         s.push(-3);
8         s.push(17);
9         System.out.println("nilai stack= "+s);
10        System.out.println("nilai pop= "+s.pop());
11        System.out.println("nilai stack setelah pop= "+s);
12        System.out.println("nilai peek="+s.peek());
13        System.out.println("nilai stack setelah peak= "+s);
14    }
15 }
16
17
18 }
```

- Pembahasan

Program ini dibuat dengan mendefinisikan sebuah class bernama LatihanStack_2511531014 yang berfungsi sebagai program utama untuk menguji penggunaan struktur data Stack.

1. Pada method main, dibuat objek Stack<Integer> s yang digunakan untuk menyimpan data bertipe Integer. Stack merupakan struktur data yang menerapkan prinsip LIFO (*Last In First Out*).

```
public class LatihanStack_2511531014 {
    public static void main (String[] args) {
        Stack<Integer> s = new Stack<Integer>();
```

2. Method `push()` digunakan untuk menambahkan elemen ke dalam stack. Pada program ini, nilai 42, -3, dan 17 dimasukkan ke dalam stack secara berurutan sehingga elemen terakhir (17) berada di posisi teratas. Perintah `System.out.println("nilai stack= " + s)` digunakan untuk menampilkan seluruh isi stack. Output akan menampilkan elemen dalam urutan dari bawah ke atas.

```
Stack<Integer> s = new Stack<Integer>();  
s.push(42);  
s.push(-3);  
s.push(17);  
System.out.println("nilai stack= "+s);
```

3. Method `pop()` digunakan untuk mengambil sekaligus menghapus elemen teratas dari stack. Nilai yang diambil kemudian ditampilkan menggunakan `System.out.println()`. Perintah `System.out.println("nilai stack setelah pop= " + s)` digunakan untuk menampilkan kondisi stack setelah elemen teratas dihapus.

```
System.out.println("nilai pop= "+s.pop());  
System.out.println("nilai stack setelah pop= "+s);
```

4. Method `peek()` digunakan untuk melihat elemen teratas tanpa menghapusnya dari stack. Nilai ini ditampilkan untuk mengetahui isi teratas setelah operasi `pop()`. Perintah `System.out.println("nilai stack setelah peek= " + s)` digunakan untuk menunjukkan bahwa isi stack tidak berubah setelah operasi `peek()`.

```
System.out.println("nilai peek"+s.peek());  
System.out.println("nilai stack setelah peak= "+s);  
  
}  
  
}
```

5. Output Hasilnya

```
Console X
<terminated> LatihanStack_2511531014 [Java Application] C:\Users\
nilai stack= [42, -3, 17]
nilai pop= 17
nilai stack setelah pop= [42, -3]
nilai peek= 3
nilai stack setelah peak= [42, -3]
```

1.2 NilaiMaksimum_2511531014

- **Kode Program**

```
1 package pekan3_2511531014;
2 import java.util.Stack;
3
4 public class NilaiMaksimum_2511531014 {
5     public static int max_2511531014(Stack<Integer> s) {
6         Stack<Integer> backup = new Stack<Integer>();
7         int maxValue_2511531014 = s.pop();
8         backup.push(maxValue_2511531014);
9         while (!s.isEmpty()) {
10             int next_2511531014 = s.pop();
11             backup.push(next_2511531014);
12             maxValue_2511531014 = Math.max(maxValue_2511531014, next_2511531014);
13         }
14         while (!backup.isEmpty()) {
15             s.push(backup.pop());
16         }
17         return maxValue_2511531014;
18     }
19     public static void main(String[] args) {
20         Stack<Integer> s = new Stack<Integer> ();
21         s.push(70);
22         s.push(12);
23         s.push(30);
24         System.out.println("Isi stack " +s);
25         System.out.println("Stack Teratas " +s.peek());
26         System.out.println("Nilai Maksimum " +max_2511531014(s));
27     }
28 }
29
30 }
```

- **Pembahasan**

Program ini dibuat dengan mendefinisikan sebuah class bernama NilaiMaksimum_2511531014 yang berfungsi untuk mencari nilai maksimum dari data yang tersimpan dalam Stack.

1. Method `max_2511531014(Stack<Integer> s)` digunakan untuk mencari nilai maksimum dari elemen yang ada di dalam stack. Method ini menerima parameter berupa objek Stack bertipe Integer. Di dalam method tersebut, dibuat sebuah Stack baru bernama backup yang digunakan untuk menyimpan sementara elemen-elemen dari stack utama agar data tidak hilang setelah proses pencarian maksimum.

```
public class NilaiMaksimum_2511531014 {
    public static int max_2511531014(Stack<Integer> s) {
        Stack<Integer> backup = new Stack<Integer>();
```

2. Variabel max_2511531014 digunakan untuk menyimpan nilai maksimum sementara. Nilai awalnya diambil dari elemen teratas stack menggunakan method pop().

```
int max_2511531014 = s.pop();
```

3. Elemen pertama yang diambil dimasukkan ke dalam stack backup menggunakan method push() agar tetap dapat dikembalikan ke stack utama setelah proses selesai.

```
backup.push(max_2511531014);
```

4. Struktur perulangan while (!s.isEmpty()) digunakan untuk memproses seluruh elemen dalam stack hingga kosong. Pada setiap iterasi, elemen diambil menggunakan pop(), kemudian disimpan ke dalam backup.

```
backup.push(max_2511531014);
while (!s.isEmpty()) {
    int next_2511531014 = s.pop();
    backup.push(next_2511531014);
```

5. Pada setiap pengambilan elemen, dilakukan perbandingan menggunakan Math.max() untuk menentukan nilai maksimum antara nilai sebelumnya dengan elemen yang baru diambil.

```
max_2511531014 = Math.max(max_2511531014, next_2511531014);
}
```

6. Setelah semua elemen diproses, dilakukan perulangan kedua while (!backup.isEmpty()) yang digunakan untuk mengembalikan semua elemen dari stack backup ke stack utama s sehingga kondisi stack kembali seperti semula.

```
}
while(!backup.isEmpty()) {
    s.push(backup.pop());
}
```

7. Method max_2511531014 kemudian mengembalikan nilai maksimum yang telah ditemukan.

```
return max_2511531014;
}
```


8. Pada method main, dibuat objek Stack<Integer> s yang digunakan untuk menyimpan data berupa bilangan bulat.

```
public static void main(String[] args) {  
    Stack<Integer> s = new Stack<Integer> ();  
    s.push(70);  
}
```

9. Method push() digunakan untuk menambahkan elemen ke dalam stack, yaitu 70, 12, dan 20.

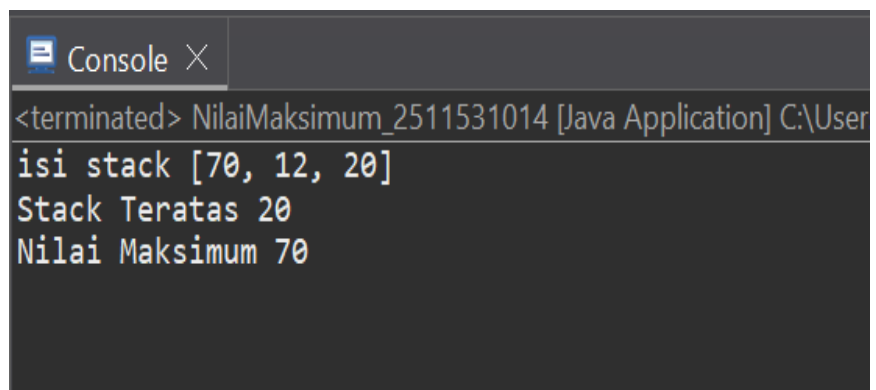
```
s.push(70);  
s.push(12);  
s.push(20);
```

10. Perintah

- System.out.println("isi stack " + s) digunakan untuk menampilkan seluruh isi stack.
- System.out.println("Stack Teratas " + s.peek()) digunakan untuk menampilkan elemen teratas tanpa menghapusnya.
- System.out.println("Nilai Maksimum " + max_2511531014(s)) digunakan untuk menampilkan nilai maksimum dari seluruh elemen dalam stack dengan memanggil method yang telah dibuat.

```
s.push(20);  
System.out.println("isi stack " + s);  
System.out.println("Stack Teratas " + s.peek());  
System.out.println("Nilai Maksimum " + max_2511531014(s));  
}  
}
```

11. Hasil Output



```
<terminated> NilaiMaksimum_2511531014 [Java Application] C:\User  
isi stack [70, 12, 20]  
Stack Teratas 20  
Nilai Maksimum 70
```

1.3 SiswaStack_2511531014

- **Kode Program**

```

1 package pekan3_2511531014;
2 import java.util.ArrayList;
3
4 class Siswa_2511531014 {
5     String nama;
6     int nim;
7
8     public Siswa_2511531014(String nama, int nim) {
9         this.nama = nama;
10        this.nim = nim;
11    }
12
13    @Override
14    public String toString() {
15        return "Nim: " + nim + ", Nama: " + nama;
16    }
17 }
18
19 public class SiswaStack_2511531014 {
20     private ArrayList<Siswa_2511531014> stack;
21
22     public SiswaStack_2511531014() {
23         stack = new ArrayList<>();
24     }
25
26     public void push(Siswa_2511531014 mhs) {
27         stack.add(mhs);
28     }
29
30     public Siswa_2511531014 pop() {
31         if (!isEmpty()) {
32             return stack.remove(stack.size() - 1);
33         }
34         return null;
35     }
36
37     public Siswa_2511531014 peek() {
38         if (!isEmpty()) {
39             return stack.get(stack.size() - 1);
40         }
41         return null;
42     }
43
44     public boolean isEmpty() {
45         return stack.isEmpty();
46     }
47
48     public void tampilkanSiswa() {
49         for (int i = stack.size() - 1; i >= 0; i--) {
50             System.out.println(stack.get(i));
51         }
52     }
53
54     public static void main(String[] args) {
55         SiswaStack_2511531014 studentStack_2511531014 = new SiswaStack_2511531014();
56
57         Siswa_2511531014 mhs1 = new Siswa_2511531014("Ali", 1);
58         Siswa_2511531014 mhs2 = new Siswa_2511531014("Boby", 2);
59         Siswa_2511531014 mhs3 = new Siswa_2511531014("Charles", 3);
60
61         studentStack_2511531014.push(mhs1);
62         studentStack_2511531014.push(mhs2);
63         studentStack_2511531014.push(mhs3);
64
65         System.out.println("Siswa di dalam stack:");
66         studentStack_2511531014.tampilkanSiswa();
67
68         System.out.println("Siswa teratas: " + studentStack_2511531014.peek());
69         System.out.println("Mengeluarkan siswa: " + studentStack_2511531014.pop());
70
71         System.out.println("Setelah pop:");
72         studentStack_2511531014.tampilkanSiswa();
73     }
74 }

```

• Pembahasan

1. Class Siswa_2511531014 digunakan untuk merepresentasikan objek siswa yang memiliki atribut nama dan nim. Constructor digunakan untuk

menginisialisasi nilai atribut saat objek dibuat, sedangkan method toString() dioverride untuk menampilkan data siswa dalam bentuk string agar mudah dibaca saat dicetak.

```
class Siswa_2511531014 {  
    String nama;  
    int nim;  
  
    public Siswa_2511531014(String nama, int nim) {  
        this.nama = nama;  
        this.nim = nim;  
    }  
  
    @Override  
    public String toString() {  
        return "Nim: " + nim + ", Nama: " + nama;  
    }  
}
```

2. Class SiswaStack_2511531014 digunakan untuk mengimplementasikan konsep Stack dengan memanfaatkan ArrayList<Siswa_2511531014> sebagai media penyimpanan. Method push() digunakan untuk menambahkan data ke bagian akhir list sebagai elemen teratas, method pop() digunakan untuk mengambil sekaligus menghapus elemen terakhir, method peek() digunakan untuk melihat elemen teratas tanpa menghapusnya, serta method isEmpty() digunakan untuk mengecek apakah stack dalam kondisi kosong.

```

9 public class SiswaStack_2511531014 {
10     private ArrayList<Siswa_2511531014> stack;
11
12     public SiswaStack_2511531014() {
13         stack = new ArrayList<>();
14     }
15
16     public void push(Siswa_2511531014 mhs) {
17         stack.add(mhs);
18     }
19
20     public Siswa_2511531014 pop() {
21         if (!isEmpty()) {
22             return stack.remove(stack.size() - 1);
23         }
24         return null;
25     }
26
27     public Siswa_2511531014 peek() {
28         if (!isEmpty()) {
29             return stack.get(stack.size() - 1);
30         }
31         return null;
32     }
33
34     public boolean isEmpty() {
35         return stack.isEmpty();
36     }
37 }

```

3. Method tampilkanSiswa() digunakan untuk menampilkan seluruh isi stack dengan perulangan dari indeks terakhir ke awal, sehingga urutan yang ditampilkan sesuai dengan konsep Stack (LIFO), yaitu dari elemen teratas ke elemen terbawah.

```

38     public void tampilkanSiswa() {
39         for (int i = stack.size() - 1; i >= 0; i--) {
40             System.out.println(stack.get(i));
41         }
42     }

```

4. Pada method main, dibuat beberapa objek siswa kemudian dimasukkan ke dalam stack menggunakan method push(). Data yang tersimpan ditampilkan menggunakan tampilkanSiswa(), kemudian elemen teratas ditampilkan menggunakan peek(), dan elemen tersebut dikeluarkan menggunakan pop().

```

public static void main(String[] args) {
    SiswaStack_2511531014 studentStack_2511531014 = new SiswaStack_2511531014();

    Siswa_2511531014 mhs1 = new Siswa_2511531014("Ali", 1);
    Siswa_2511531014 mhs2 = new Siswa_2511531014("Boby", 2);
    Siswa_2511531014 mhs3 = new Siswa_2511531014("Charles", 3);

    studentStack_2511531014.push(mhs1);
    studentStack_2511531014.push(mhs2);
    studentStack_2511531014.push(mhs3);

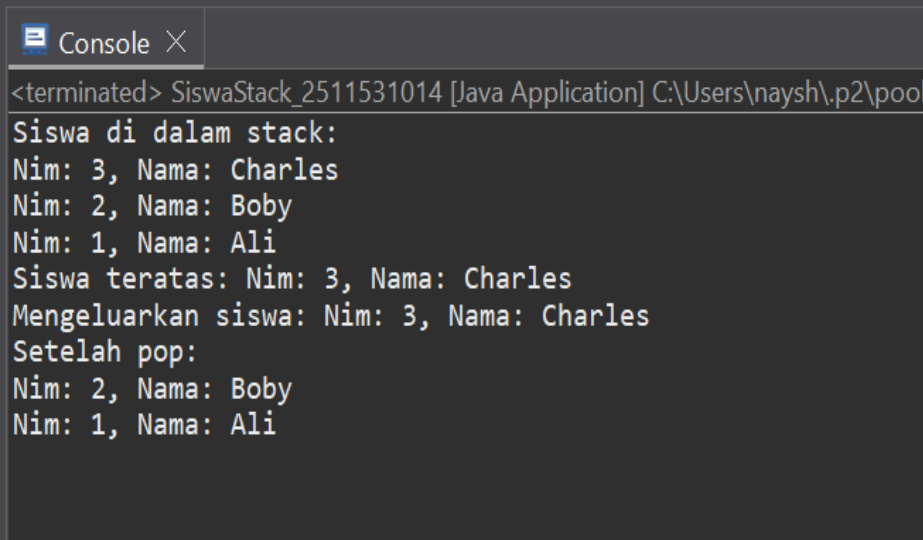
    System.out.println("Siswa di dalam stack:");
    studentStack_2511531014.tampilkanSiswa();

    System.out.println("Siswa teratas: " + studentStack_2511531014.peek());
    System.out.println("Mengeluarkan siswa: " + studentStack_2511531014.pop());

    System.out.println("Setelah pop:");
    studentStack_2511531014.tampilkanSiswa();
}
}

```

5. Output hasil berupa



```

<terminated> SiswaStack_2511531014 [Java Application] C:\Users\naysh\.p2\pool
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
Siswa teratas: Nim: 3, Nama: Charles
Mengeluarkan siswa: Nim: 3, Nama: Charles
Setelah pop:
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali

```

1.4 StackArray_2511531014

- Kode Program

```

1 package pekan3_2511531014;
2
3 public class StackArray_2511531014 {
4     static final int MAX = 1000;
5     int top;
6     int a[] = new int [MAX];
7     boolean isEmpty_2511531014()
8     {
9         return (top < 0);
10    }
11    StackArray_2511531014()
12    {
13        top = -1;
14    }
15    boolean push_2511531014 (int x)
16    {
17        if (top >= (MAX - 1)) {
18            System.out.println("Stack Overflow");
19            return false;
20        }
21        else {
22            a[++top] = x;
23            System.out.println(x + " dimasukkan dalam stack");
24            return true;
25        }
26    }
27
28    int pop_2511531014()
29    {
30        if (top < 0) {
31            System.out.println ("Stack Underflow");
32            return 0;
33        }
34
35        else {
36            int x = a[top--];
37            return x;
38        }
39    }
40    int peek_2511531014()
41    {
42        if (top < 0) {
43            System.out.println("StackUnderflow");
44            return 0;
45        }
46        else {
47            int x = a[top];
48            return x;
49        }
50    }
51
52    void print_2511531014() {
53        for (int i = top; i >= 0; i--) {
54            System.out.print(" " + a[i]);
55        }
56    }
57
58 }

```

- **Pembahasan**

Program ini dibuat dengan mendefinisikan class StackArray_2511531014 yang digunakan untuk mengimplementasikan struktur data Stack menggunakan array. Pada bagian awal, ditentukan konstanta MAX sebagai kapasitas maksimum stack, serta atribut top yang digunakan untuk menunjukkan posisi elemen teratas, dan array a[] sebagai media penyimpanan data.

1. Constructor StackArray_2511531014() digunakan untuk menginisialisasi nilai top menjadi -1 yang menandakan bahwa stack dalam kondisi kosong. Method isEmpty_2511531014() digunakan untuk mengecek apakah stack kosong dengan cara memeriksa nilai top.

```

2
3 public class StackArray_2511531014 {
4     static final int MAX = 1000;
5     int top;
6     int a[] = new int [MAX];
7     boolean isEmpty_2511531014()
8     {
9         return (top < 0);
10    }
11    StackArray_2511531014()
12    {
13        top = -1;
14    }

```

2. Method `push_2511531014(int x)` digunakan untuk menambahkan elemen ke dalam stack. Sebelum penambahan, dilakukan pengecekan kondisi overflow ketika stack telah mencapai kapasitas maksimum. Jika masih tersedia ruang, nilai `top` akan dinaikkan dan elemen dimasukkan ke dalam array, kemudian ditampilkan pesan bahwa data berhasil ditambahkan.

```

5     boolean push_2511531014 (int x)
6     {
7         if (top >= (MAX - 1)) {
8             System.out.println("Stack Overflow");
9             return false;
10        }
11        else {
12            a[++top] = x;
13            System.out.println(x + " dimasukkan dalam stack");
14            return true;
15        }
16    }

```

3. Method `pop_2511531014()` digunakan untuk mengambil sekaligus menghapus elemen teratas dari stack. Method ini melakukan pengecekan kondisi underflow jika stack kosong. Jika tidak kosong, elemen teratas diambil dari array dan nilai `top` dikurangi satu.

```

5     int pop_2511531014()
6     {
7         if (top < 0) {
8             System.out.println ("Stack Underflow");
9             return 0;
10        }
11        else {
12            int x = a[top--];
13            return x;
14        }
15    }

```

4. Method `peek_2511531014()` digunakan untuk melihat elemen teratas tanpa menghapusnya. Method ini juga melakukan pengecekan apakah stack kosong sebelum mengakses data, sehingga dapat menghindari kesalahan.

```
int peek_2511531014()
{
    if (top < 0) {
        System.out.println("StackUnderflow");
        return 0;
    }
    else {
        int x = a[top];
        return x;
    }
}
```

5. Method `print_2511531014()` digunakan untuk menampilkan seluruh isi stack dengan menggunakan perulangan dari indeks top hingga ke indeks 0, sehingga urutan yang ditampilkan sesuai dengan konsep Stack (LIFO), yaitu dari elemen teratas ke elemen terbawah.

```
void print_2511531014() {
    for (int i = top; i >= 0; i--) {
        System.out.print(" " + a[i]);
    }
}
```

1.5 StackArrayDriver_2511531014

1. Kode Program

```
1 package pekan3_2511531014;
2
3 public class StackArrayDriver_2511531014 {
4     public static void main(String[] args) {
5         StackArray_2511531014 s = new StackArray_2511531014();
6         s.push_2511531014(10);
7         s.push_2511531014(20);
8         s.push_2511531014(30);
9         System.out.println(s.pop_2511531014() + " dikeluarkan dari stack");
10        System.out.println("Elemen teratas adalah : " + s.peek_2511531014());
11        System.out.println(" element pada stack :");
12        s.print_2511531014();
13    }
14 }
15 }
16 }
```

• Pembahasan

Program ini dibuat dengan mendefinisikan class `StackArrayDriver_2511531014` yang berfungsi sebagai driver untuk

menguji implementasi struktur data Stack berbasis array pada class StackArray_2511531014. Pada method main, dibuat objek StackArray_2511531014 s yang digunakan untuk melakukan berbagai operasi pada stack.

1. Method push_2511531014() digunakan untuk menambahkan elemen ke dalam stack, yaitu nilai 10, 20, dan 30 secara berurutan, sehingga elemen terakhir yang dimasukkan berada pada posisi teratas sesuai prinsip LIFO (*Last In First Out*).

```
public class StackArrayDriver_2511531014 {  
    public static void main(String[] args) {  
        StackArray_2511531014 s = new StackArray_2511531014();  
        s.push_2511531014(10);  
        s.push_2511531014(20);  
        s.push_2511531014(30);
```

2. Method pop_2511531014() digunakan untuk mengambil sekaligus menghapus elemen teratas dari stack, dan hasilnya ditampilkan menggunakan System.out.println() sebagai elemen yang dikeluarkan dari stack.

```
        System.out.println(s.pop_2511531014() + " dikeluarkan dari stack");
```

3. Method peek_2511531014() digunakan untuk melihat elemen teratas setelah proses pop() tanpa menghapusnya, sehingga dapat diketahui nilai yang berada di posisi paling atas saat ini.

```
        System.out.println("Elemen teratas adalah : " + s.peek_2511531014());  
        System.out.println(" element pada stack :");
```

4. Method print_2511531014() digunakan untuk menampilkan seluruh isi stack dengan urutan dari elemen teratas ke elemen terbawah, sehingga sesuai dengan konsep kerja stack.

```
        s.print_2511531014();  
    }  
}
```

5. Output yang di hasilkan

```
Console X
<terminated> StackArrayDriver_2511531014 [Java Application] C:\Us
10 dimasukan dalam stack
20 dimasukan dalam stack
30 dimasukan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
element pada stack :
20 10
```

1.6 StackPostfix_2511531014

2. Kode Program

```
1 package pekan3_2511531014;
2 import java.util.Scanner;
3 import java.util.Stack;
4
5 public class StackPostfix_2511531014 {
6     public static int postfixEvaluate(String expression) {
7         Stack<Integer> s= new Stack<Integer>();
8         Scanner input = new Scanner(expression);
9         while (input.hasNext()) {
10             if (input.hasNextInt()) { // an operand (integer)
11                 s.push(input.nextInt());
12             } else { // an operator
13                 String operator = input.next();
14                 int operand2 = s.pop();
15                 int operand1 = s.pop();
16                 if (operator.equals("+")) {
17                     s.push(operand1 + operand2);
18                 } else if (operator.equals("-")) {
19                     s.push(operand1 - operand2);
20                 } else if (operator.equals("*")) {
21                     s.push(operand1 * operand2);
22                 } else {
23                     s.push(operand1 / operand2);
24                 }
25             }
26         }
27         input.close();
28         return s.pop();
29     }
30
31     public static void main (String [] args) {
32         System.out.println ("hasil Postfix_2511531014= " + postfixEvaluate ("5 2 4 * + 7 -"));
33     }
34 }
```

• Pembahasan

Program ini dibuat dengan mendefinisikan class StackPostfix_2511531014 yang berfungsi untuk menghitung hasil dari ekspresi postfix.

1. Method postfixEvaluate(String expression) digunakan untuk mengevaluasi ekspresi postfix. Di dalam method ini dibuat objek Stack<Integer> sebagai tempat penyimpanan operand, serta objek Scanner yang digunakan untuk membaca setiap token dalam ekspresi. Jika token

berupa bilangan (hasNextInt()), maka nilai tersebut dimasukkan ke dalam stack menggunakan push(). Jika token berupa operator, maka dua elemen teratas diambil menggunakan pop() sebagai operand, kemudian dilakukan operasi sesuai operator seperti penjumlahan, pengurangan, perkalian, atau pembagian, dan hasilnya dimasukkan kembali ke dalam stack.

```
public class StackPostfix_2511531014 {  
    public static int postfixEvaluate (String expression) {  
        Stack<Integer> s= new Stack<Integer>();  
        Scanner input = new Scanner(expression);
```

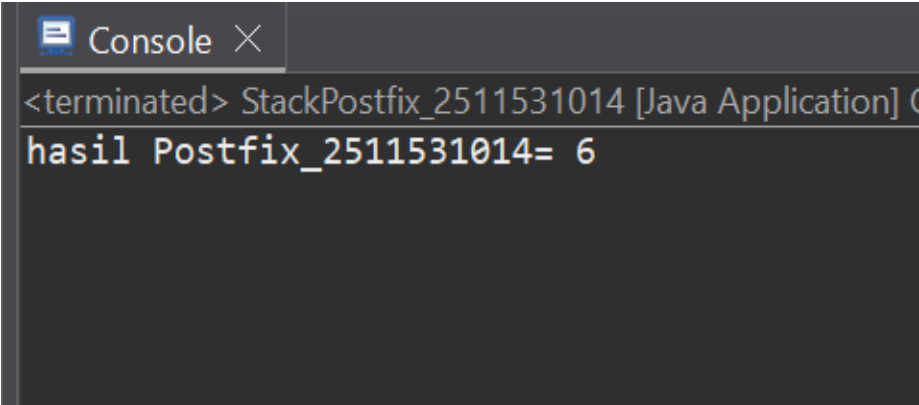
2. Proses evaluasi dilakukan menggunakan perulangan while (input.hasNext()) hingga seluruh ekspresi selesai dibaca. Setelah semua operasi selesai, nilai terakhir yang tersisa di dalam stack merupakan hasil akhir dari perhitungan dan dikembalikan sebagai nilai return.

```
31 public static void main (String [] args) {  
32     System.out.println ("hasil Postfix_2511531014= " + postfixEvaluate ("5 2 4 * + 7 -"));  
33 }  
34 }
```

3. Pada method main, program memanggil method postfixEvaluate() dengan input ekspresi postfix "5 2 4 * + 7 -", kemudian hasil perhitungan ditampilkan menggunakan System.out.println().

```
while (input.hasNext()) {  
    if (input.hasNextInt()) { // an operand (integer)  
        s.push(input.nextInt());  
    } else { // an operator  
        String operator = input.next();  
        int operand2 = s.pop();  
        int operand1 = s.pop();  
        if (operator.equals("+")) {  
            s.push(operand1 + operand2);  
        } else if (operator.equals("-")) {  
            s.push(operand1 - operand2);  
        } else if (operator.equals("*")) {  
            s.push(operand1 * operand2);  
        } else {  
            s.push(operand1 / operand2);  
        }  
    }  
}  
input.close();  
return s.pop();  
}
```

4. Hasil Outputnya



The screenshot shows a console window titled "Console" with a close button. The output text is: `<terminated> StackPostfix_2511531014 [Java Application] C
hasil Postfix_2511531014= 6`

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi struktur data Stack menggunakan array dan ArrayList telah berhasil dilakukan dengan baik serta mampu merepresentasikan konsep Abstract Data Type (ADT) Stack. Kedua metode implementasi tersebut mendukung operasi dasar Stack, seperti penambahan data (*push*), penghapusan data (*pop*), serta melihat elemen teratas (*peek*), yang semuanya bekerja berdasarkan prinsip LIFO (*Last In First Out*).

Pada implementasi menggunakan array, struktur data memiliki ukuran tetap sehingga memerlukan pengelolaan indeks secara manual serta penanganan kondisi overflow dan underflow. Hal ini membuat penggunaan array lebih efisien dari segi memori, namun kurang fleksibel karena kapasitasnya terbatas.

Sedangkan pada implementasi menggunakan ArrayList, struktur data bersifat dinamis sehingga tidak memiliki batasan ukuran tetap dan lebih mudah dalam pengelolaan data. Operasi seperti penambahan dan penghapusan elemen dapat dilakukan dengan lebih fleksibel tanpa perlu mengatur kapasitas secara manual, meskipun terdapat sedikit overhead dibandingkan array.

Melalui berbagai program yang telah dibuat, seperti Stack berbasis array, Stack menggunakan ArrayList, serta penerapan Stack dalam kasus nyata seperti pencarian nilai maksimum dan evaluasi ekspresi postfix, dapat disimpulkan bahwa struktur data Stack sangat berguna dalam menyelesaikan berbagai permasalahan pemrograman secara sistematis.

Secara keseluruhan, praktikum ini memberikan pemahaman yang lebih mendalam mengenai konsep Stack, perbedaan implementasi menggunakan array dan ArrayList, serta penerapannya dalam pemrograman Java, sehingga dapat meningkatkan kemampuan dalam merancang dan mengelola struktur data secara efisien dan terstruktur.

TUGAS PEKAN 3

Topik : Implementasi Stack (LIFO) menggunakan java.util.Stack

Tema : Simulasi Browser History

Terdiri dari 2 Class yaitu Class Website dan Class Driver(Browser)

1. Website_2511531014

• Kode Program

```
1 package tugasPekan3_2511531014;
2 import java.util.Scanner;
3 import java.util.Stack;
4
5 public class Browser_2511531014 {
6     public static void main(String[] args) {
7
8         Scanner input = new Scanner(System.in);
9
10        Stack<Website_2511531014> history = new Stack<>();
11        int pilihan;
12        do {
13            // Menu pilihan program browser
14            System.out.println("\n=== BROWSER HISTORY 2511531014 ===");
15            System.out.println("1. Kunjungi Website (Push)");
16            System.out.println("2. Tombol Back (Pop)");
17            System.out.println("3. Lihat Halaman Aktif (Peek)");
18            System.out.println("4. Keluar");
19            System.out.print("Pilihan: ");
20            pilihan = input.nextInt();
21            input.nextLine();
22
23            switch (pilihan) {
24                case 1:
25                    // Push → menambahkan website ke tumpukan atau stack
26                    System.out.print("Masukkan Judul Website: ");
27                    String judul = input.nextLine();
28
29                    System.out.print("Masukkan URL: ");
30                    String url = input.nextLine();
31
32                    history.push(new Website_2511531014(judul, url));
33
34                    System.out.println("Berhasil mengunjungi website!");
35                    break;
36
37                case 2:
38                    // Pop → menghapus data terakhir dikunjungi
39                    if (!history.isEmpty()) {
40                        Website_2511531014 w = history.pop();
41                        System.out.println("Kembali dari: " + w.getJudul());
42                    } else {
43                        System.out.println("History kosong!");
44                    }
45                    break;
46
47                case 3:
48                    // Peek → melihat data paling atas atau yang sedang di kunjungi tanpa menghapus
49                    if (!history.isEmpty()) {
50                        Website_2511531014 w = history.peek();
51                        System.out.println("Halaman aktif:");
52                        System.out.println(w.getJudul() + " - " + w.getUrl());
53                    } else {
54                        System.out.println("Tidak ada halaman!");
55                    }
56                    break;
57
58                case 4:
59                    System.out.println("Keluar Halaman.");
60                    break;
61
62                default:
63                    System.out.println("Pilihan tidak valid!");
64            }
65        } while (pilihan != 4);
66    }
67 }
```

• Pembahasan

1. Pada method main, dibuat objek Scanner input yang digunakan untuk menerima input dari pengguna, serta Stack<Website_2511531014> history yang digunakan untuk menyimpan data website dalam bentuk

objek secara berurutan. Struktur data Stack digunakan karena menerapkan prinsip LIFO (*Last In First Out*), yang sesuai dengan konsep navigasi browser.

```
Stack<Website_2511531014> history = new Stack<>();
int pilihan;
```

2. Program menggunakan perulangan do-while untuk menampilkan menu secara berulang hingga pengguna memilih keluar. Menu yang disediakan meliputi kunjungi website (*push*), kembali ke halaman sebelumnya (*pop*), melihat halaman aktif (*peek*), dan keluar dari program.

```
do {
    // Menu pilihan program browser
    System.out.println("\n=== BROWSER HISTORY 2511531014 ===");
    System.out.println("1. Kunjungi Website (Push)");
    System.out.println("2. Tombol Back (Pop)");
    System.out.println("3. Lihat Halaman Aktif (Peek)");
    System.out.println("4. Keluar");
    System.out.print("Pilihan: ");
    pilihan = input.nextInt();
    input.nextLine();
}
```

3. Pada pilihan pertama, method `push()` digunakan untuk menambahkan data website ke dalam stack. Data berupa judul dan URL diinput oleh pengguna, kemudian dibuat menjadi objek `Website_2511531014` dan dimasukkan ke dalam stack sebagai halaman yang sedang dikunjungi.

```
switch (pilihan) {
    case 1:
        // Push → menambahkan website ke tumpukan atau stack
        System.out.print("Masukkan Judul Website: ");
        String judul = input.nextLine();

        System.out.print("Masukkan URL: ");
        String url = input.nextLine();

        history.push(new Website_2511531014(judul, url));

        System.out.println("Berhasil mengunjungi website!");
        break;
```

4. Pada pilihan kedua, method `pop()` digunakan untuk menghapus elemen teratas dari stack yang merepresentasikan tombol *back* pada browser. Sebelum penghapusan, dilakukan pengecekan

menggunakan isEmpty() untuk memastikan bahwa stack tidak kosong.

```
break;
case 2:
    // Pop → menghapus data terakhir dikunjungi
    if (!history.isEmpty()) {
        Website_2511531014 w = history.pop();
        System.out.println("Kembali dari: " + w.getJudul());
    } else {
        System.out.println("History kosong!");
    }
    break;
```

5. Pada pilihan ketiga, method peek() digunakan untuk melihat elemen teratas tanpa menghapusnya, sehingga dapat menampilkan halaman yang sedang aktif. Pengecekan isEmpty() juga digunakan untuk menghindari kesalahan saat stack kosong.

```
break;
case 3:
    // Peek → melihat data paling atas atau yang sedang di kunjungi tanpa menghapus
    if (!history.isEmpty()) {
        Website_2511531014 w = history.peek();
        System.out.println("Halaman aktif:");
        System.out.println(w.getJudul() + " - " + w.getUrl());
    } else {
        System.out.println("Tidak ada halaman!");
    }
    break;
```

6. Pada pilihan keempat, program akan berhenti dengan menampilkan pesan keluar. Selain itu, kondisi default digunakan untuk menangani input yang tidak sesuai dengan pilihan menu.

```
break;
case 4:
    System.out.println("Keluar Halaman.");
    break;
default:
    System.out.println("Pilihan tidak valid!");
}
} while (pilihan != 4);
}
```

Dilanjutkan dengan Class Driver

2. Browser_2511531014

- Kode Program

```

1 package tugasPekan5_2511531014;
2
3 class Website_2511531014 {
4
5     String judul_1014;
6     String url_1014;
7
8     // Constructor untuk menginisialisasi objek
9     public Website_2511531014(String judul, String url) {
10         this.judul_1014 = judul;
11         this.url_1014 = url;
12     }
13
14     // Getter untuk mengambil nilai atribut
15     public String getJudul() {
16         return judul_1014;
17     }
18     public String getUrl() {
19         return url_1014;
20     }
21
22     // Setter untuk mengubah nilai atribut
23     public void setJudul(String judul) {
24         this.judul_1014 = judul;
25     }
26     public void setUrl(String url) {
27         this.url_1014 = url;
28     }
29 }

```

• Pembahasan

Program ini dibuat dengan mendefinisikan class Website_2511531014 yang digunakan untuk merepresentasikan objek website dalam program browser history. Class ini berfungsi sebagai model data yang menyimpan informasi terkait website.

1. Class ini memiliki dua atribut bertipe String, yaitu judul_1014 dan url_1014 yang digunakan untuk menyimpan judul website dan alamat URL. Kedua atribut ini menjadi identitas utama dari setiap objek website yang disimpan dalam Stack.

```

2
3 class Website_2511531014 {
4
5     String judul_1014;
6     String url_1014;
7

```

2. Constructor Website_2511531014(String judul, String url) digunakan untuk menginisialisasi nilai atribut saat objek dibuat. Di dalam constructor digunakan keyword this seperti pada this.judul_1014 = judul dan this.url_1014 = url untuk membedakan antara parameter dan atribut class.


```

7
8 // Constructor untuk menginisialisasi objek
9 public Website_2511531014(String judul, String url) {
10     this.judul_1014 = judul;
11     this.url_1014 = url;
12 }
13

```

3. Method getter yaitu getJudul() dan getUrl() digunakan untuk mengambil nilai dari atribut judul_1014 dan url_1014 sehingga data dapat diakses dari luar class tanpa harus mengakses atribut secara langsung.

```

13
14 // Getter untuk mengambil nilai atribut
15 public String getJudul() {
16     return judul_1014;
17 }
18 public String getUrl() {
19     return url_1014;
20 }
21

```

4. Method setter yaitu setJudul(String judul) dan setUrl(String url) digunakan untuk mengubah nilai atribut yang sudah ada, sehingga data website dapat diperbarui sesuai kebutuhan.

```

21
22 // Setter untuk mengubah nilai atribut
23 public void setJudul(String judul) {
24     this.judul_1014 = judul;
25 }
26 public void setUrl(String url) {
27     this.url_1014 = url;
28 }
29 }

```

5. Hasil Outputnya

- Kunjungi Website (Push)

```
=== BROWSER HISTORY 2511531014 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 1
Masukkan Judul Website: iLearn
Masukkan URL: https://it.ilearn.unand.ac.id
Berhasil mengunjungi website!

=== BROWSER HISTORY 2511531014 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 1
Masukkan Judul Website: SSO UNAND
Masukkan URL: https://sso.unand.ac.id
Berhasil mengunjungi website!
```

- Kembali dari Halaman yang sedang dibuka (Pop)

```
=== BROWSER HISTORY 2511531014 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 2
Kembali dari: SSO UNAND
```

- Halaman yang sedang terbuka (Peek)

```
=== BROWSER HISTORY 2511531014 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 3
Halaman aktif:
iLearn - https://it.ilearn.unand.ac.id
```

- Keluar Dari Halaman

```
=== BROWSER HISTORY 2511531014 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 4
Keluar Halaman.
```